

个人中心与充值功能安全修复报告

Flask 用户管理系统

实验名称：个人中心与充值功能安全修复

实验日期：2026 年 7 月 10 日

1 项目概述

1.1 项目简介

本项目是一个基于 Python Flask 的用户信息管理系统，已提供登录、注册、搜索、上传头像等功能。本次新增个人中心和充值功能，并对其中存在的安全漏洞进行分析和修复。

1.2 新增功能接口

路由	方法	功能
GET /profile	GET	显示当前登录用户的个人资料
POST /recharge	POST	对当前登录用户的余额进行充值

1.3 本次发现的漏洞数量

经过对新增功能的代码审计，共发现 3 处 安全漏洞：

编号	漏洞名称	位置	风险等级
AUTH-001	越权访问 — 未校验当前用户身份	/profile 路由	高危
Sqli-004	SQL 注入 — f-string 拼接 user_id	/profile + /recharge 路由	严重
BIZ-001	业务逻辑漏洞 — 未校验 amount 正负	/recharge 路由	中危

2 漏洞分析

2.1 漏洞 AUTH-001：越权访问

2.1.1 漏洞位置

文件	app.py
函数	profile()
路由	GET /profile

2.1.2 修复前代码

```
@app.route("/profile")
def profile():
    if "username" not in session:
        return redirect("/login")
    user_id = request.args.get("user_id", "")
    ...
    c.execute(f"SELECT ... FROM users WHERE id = {user_id}")
```

2.1.3 漏洞原理

个人中心路由从 URL 参数 `user_id` 获取要查询的用户 ID，没有验证当前登录用户与要查询的 `user_id` 是否匹配。攻击者可以通过修改 URL 中的 `user_id` 参数值，查看任意用户的个人资料和余额信息。例如：

攻击流程：

1. alice 登录系统
2. 浏览器访问 `/profile?user_id=1`
3. 服务器返回 admin 的用户信息（不校验权限）
4. alice 成功获取 admin 的邮箱、手机、余额

2.1.4 修复后代码

```
@app.route("/profile")
def profile():
    current_user = session.get("username")
    if not current_user:
        return redirect("/login")
    ...
    c.execute("SELECT ... FROM users WHERE username = ?", (current_user,))
```

2.1.5 为什么这样修改

不再从 URL 参数获取 `user_id`，改为从 `session["username"]` 获取当前登录用户名。查询时使用 `WHERE username = ?` 限定只返回当前用户的数据。无论攻击者如何修改 URL 参数，查询条件始终基于 session 中的用户名，无法越权查看其他用户。

2.2 漏洞 SQLI-004：SQL 注入

2.2.1 漏洞位置

`profile()` 和 `recharge()` 两个函数均存在 SQL 注入。

2.2.2 修复前代码

```
# profile 路由
c.execute(f"SELECT ... FROM users WHERE id = {user_id}")

# recharge 路由
sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
c.execute(sql)
```

2.2.3 漏洞原理

`user_id` 和 `amount` 直接通过 f-string 拼接到 SQL 语句中。攻击者可以在参数中插入 SQL 代码来改变查询逻辑。例如 `user_id = "1 OR 1=1"` 可查询全部用户，`amount = "100; DROP TABLE users"` 可执行任意 SQL 命令。

2.2.4 修复后代码

```
# profile 路由
c.execute("SELECT ... FROM users WHERE username = ?", (current_user,))

# recharge 路由
c.execute("UPDATE users SET balance = balance + ? WHERE username = ?", (amount, current_user))
```

2.2.5 为什么能防御 SQL 注入

使用 `?` 占位符将 SQL 结构与数据分离。用户输入被当作参数传入，不会成为 SQL 代码的一部分。即使 `amount` 为 `"100; DROP TABLE users"`，数据库也会将其作为一个浮点数字段值处理，而不是执行 `DROP TABLE`。

2.3 漏洞 BIZ-001：业务逻辑漏洞

2.3.1 漏洞位置

`recharge()` 函数未校验 `amount` 参数。

2.3.2 修复前代码

```
amount = request.form.get("amount", "0")
sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
```

2.3.3 漏洞原理

amount 参数直接从表单获取，不做正负校验。攻击者可以提交负数金额来减少其他用户的余额，或者提交超大数字来溢出系统。

2.3.4 修复后代码

```
try:
    amount = float(amount_str)
except ValueError:
    return "金额格式无效", 400

if amount <= 0:
    return "充值金额必须为正数", 400

if amount > 999999:
    return "单次充值金额不能超过 999999", 400
```

3 修复前后对比

对比项	修复前	修复后
用户身份获取方式	URL 参数 user_id	session["username"]
SQL 构造方式	f-string 拼接	? 占位符参数化
amount 正负校验	无校验	amount > 0 必须为正数
amount 格式校验	无校验	float() 转换，非数字报错
amount 上限校验	无上限	单次不超过 999999
越权防护	无防护	只能查看自己的资料

4 修复验证

测试项	测试方法	修复前结果	修复后结果
admin 查看个人中心	admin 登录后访问 /profile	显示 admin 信息	显示 admin 信息
alice 查看个人中心	alice 登录后访问 /profile	显示 alice 信息	显示 alice 信息
越权访问	alice 访问 /profile?user_id=1	返回 admin 信息	参数被忽略，返回自己的资料
正数充值	amount=200	余额增加 200	余额增加 200
负数充值	amount=-500	余额减少 500	拦截，提示必须为正数
非数字充值	amount=abc	SQL 报错	拦截，提示格式无效
SQL 注入充值	amount=100;DROP TABLE users	可能执行恶意 SQL	拦截，提示格式无效
URL 参数 user_id 注入	访问 /profile?user_id=1 OR 1=1	参数注入 SQL	参数被忽略，无注入

5 实验总结

5.1 漏洞产生原因

- 越权漏洞：从 URL 接收 user_id 查询用户资料，未验证当前用户身份
- **SQL** 注入：使用 f-string 将 user_id 和 amount 直接拼接到 SQL 语句
- 业务逻辑漏洞：未校验 amount 必须为正数，允许负数扣减余额

5.2 修复方法

漏洞	修复方法
AUTH-001 越权	从 session 获取当前用户名，不使用 URL 参数
SQLI-004 SQL 注入	参数化查询（? 占位符），禁止 f-string 拼接
BIZ-001 业务逻辑	校验 amount 为正数、数字格式、上限 999999

5.3 修复效果

- 越权查看其他用户资料 →  已防御
- SQL 注入攻击 →  已防御（参数化查询）
- 负数扣减余额 →  已防御（正数校验）
- 非数字/恶意 SQL 输入 →  已防御（格式校验）
- 正常个人中心查看 →  功能正常
- 正常充值 →  功能正常